

You can query the values of the *read-only attributes*, but you cannot set them as *Plant Simulation* computes the value for the point-in-time at which you query it. In most cases a *read-only attribute* corresponds to an unavailable dialog item on one of the tabs of the object, for example on the tab **Statistics**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.

Name	Value	Inherited	Watchable	Signature
NumPartsSinceSetup	0			-> integer
NumPred	1			-> integer
NumSucc	3			-> integer

Name of the read-only attribute Current value of the read-only attribute Data type of the return value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

To query the value of a *read-only attribute*, you might, for example, type:

```
print MyWorkplace.UUID
```

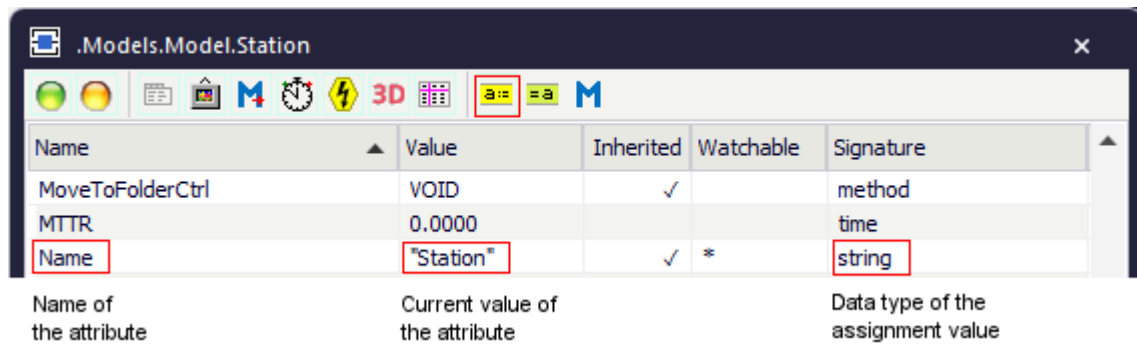
Attributes of the Workplace

_Attributes of the Workplace

The *Workplace* provides:

- The *attributes* listed in the table of contents to the left.
- The [Attributes of All Objects](#).
- The [Attributes of the Material Flow Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

- To **set the value** of an attribute, you might, for example, type:

```
MyWorkplace.Station := MyStation
```

- To **get the value** of an attribute, you might, for example, type:

```
print MyWorkplace.Station
posit := Station.Cont.XPos
```

```
print MyPythonModule.Origin
```

- To **set the value** of an attribute, you might, for example, type:
- To **get the value** of an attribute, you might, for example, type:

AtEntrance [SimTalk]

Attaches the *Workplace* designated by <Path> to the entrance of the assigned material flow object (*true*) or not (*false*).

Remarks

Assigning a new station to a *Workplace* via *SimTalk* does not change the current assignment to the entrance or the exit.

Type

Attribute

Syntax

```
<Path>.AtEntrance:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyWorkplace.AtEntrance := true
```

See also

[At Entrance \[check box\]](#)

[Model a Worker Who Waits for a Free Target](#)

AtExit [SimTalk]

Attaches the *Workplace* designated by <Path> to the exit of the assigned material flow object (*true*) or not (*false*).

Remarks

Assigning a new station to a *Workplace* via *SimTalk* does not change the current assignment to the entrance or the exit.

Type

Attribute

Syntax

```
<Path>.AtExit: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyWorkplace.AtExit := true
```

See also

[At Exit \[check box\]](#)

[Model a Worker Who Waits for a Free Target](#)

Capacity [SimTalk] - Workplace

Sets the **Capacity**, i.e., the number of *Workers* that can stay on the *Workplace* designated by <Path> at any one time.

Remarks

The **Capacity** is an integer value greater than 0. The **Capacity** is important if several tasks can be accomplished on this *Workplace* or if these tasks come together there.

Plant Simulation uses the coordinate of the *Workplace* to find the shortest route to it, meaning that the *Worker* walks to this coordinate and then jumps to its animation point.

Note

You can only change the **Capacity** if the *Workplace* is empty.

Type

Attribute

Syntax

```
<Path>.Capacity:integer
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *integer*.

Example

```
MyWorkplace.Capacity := 4
```

See also

[Capacity \[text box\] - Workplace](#)

DistanceInQueue [SimTalk]

Sets the distance between the *Workers* in the queue on the animation path of the *Workplace* designated by <Path>.

Remarks

The *Workers* attempt to keep this distance while they wait in line to step onto the *Workplace* and do their job.

Type

Attribute

Syntax

```
<Path>.DistanceInQueue:length
```

Assignment Value

You can assign a value of data type *length*.

Example

```
.Resources.Worker:1.DistanceInQueue := 1 // 1 meter
```

See also

[Distance in Queue](#)

[How Workers Queue Up in Front of the Workplace](#)

EntranceCtrl [SimTalk] - Workplace

Designates a *Method* object of the object designated by <Path>. *Plant Simulation* calls the *Method* once the *Worker* has stepped onto the *Workplace*.

Type

Attribute

Syntax

```
<Path>.EntranceCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Examples

```
MyWorkplace.EntranceCtrl := &myEntranceCtrl
```

```
// Entrance control of the workplace WP_Arrival
if (NOT Arrival.WorkerWasOrdered) AND Arrival.occupied AND
Arrival.cont.finished
  var WorkerTab:object := AssignedWorkers
  for var j := 1 to WorkerTab.yDim
    if WorkerTab[1,j] = @
      WorkerTab[2,j] := true // assigned
      Arrival.cont.move(@)
      waituntil Arrival.succ.~.WP_buffer.entranceFree
      @.goto(Arrival.succ.~.WP_buffer)
      return
    end
  next
end
```

```
// Entrance control of the workplace WP_Shipping
for var j := 1 to AssignedWorkers.yDim
  var Worker:object := AssignedWorkers[1,j]
  if Worker = @
    waituntil Shipping.empty
    AssignedWorkers[2,j] := false // unassigned
    @.cont.move(Shipping)
    waituntil WP_Arrival.entranceFree
    Worker.goto(WP_Arrival)
  exitloop
end
```

```
end  
next
```

See also

[Entrance Control \[Workplace\]](#)

ExitCtrl [SimTalk] - Workplace

Designates a *Method* object of the object designated by <Path>. *Plant Simulation* calls the *Method* once the *Worker* has stepped off the *Workplace*.

Type

Attribute

Syntax

```
<Path>.ExitCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyWorkplace.ExitCtrl := &myExitControl
```

See also

[Exit Control \[Workplace\]](#)

LoadCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

The **Load Control** sets how the part is transferred from the *work station* to the *Worker* staying on the attached *Workplace*. After picking the part up, the *Worker* carries it to the target object.

Plant Simulation calls the **Load Control** as soon as the *Worker* steps onto the *Workplace*.

The **Load Control** is also called if the *Worker* is already located on the *Workplace* and is to load and carry parts to the target object from there.

The **standard load control** as a *user-defined attribute* looks like this:

```
// workplace load control ...
// ... to transfer MUs from the associated station to the worker on the
workplace
// @: worker
// ?: workplace
if @.failed = false and @.pause = false
    // get the first MU to exit and repeat this until no MU is about to exit
    var mu:object := ?.station.cont

    while mu /= void and @.full = false
        // transfer the exiting MU to the worker
        mu.move(@)

        // with configured load time on workplace, wait for EndOfTime event
        causing the transfer to worker
        waituntil mu.~ = @

        mu := ?.station.cont
    end
end
```

Type

Attribute

Syntax

```
<Path>.LoadCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyWorkplace.LoadCtrl := &myLoadControl
```

See also

[Load Control](#)

LoadingTime [SimTalk] - Workplace

Sets the **Loading Time** of the *Workplace* designated by <Path>.

Remarks

The **Loading Time** is the time it takes the *Worker* for picking up a *part* at the *Workplace*. After the **Loading Time** has elapsed, the *Worker* waits for the **Maximum Dwell Time** to pass if he still has carrying capacity. If not, he immediately carries the *part* to the target station and places it there. If the *Worker* is to load more than one *part*, the **Maximum Dwell Time** should at least be as long as the carrying capacity times the **Loading Time**.

Use the anonymous identifier @ to access the *part* for which the loading time applies.

Note

During the **Loading Time** no *part* can be placed onto the delivering station.

Note

Behavior during interruptions of the Worker

Plant Simulation extends the **Loading Time** by **Failure Times** of the *Worker*. The processing station remains locked during the failure.

If the *Worker* **pauses** during the **Loading Time**, *Plant Simulation* cancels loading and considers it as completely elapsed. This means that the *Worker* carries at least one *part* to the *WorkerPool*. When the shift ends during the **Loading Time**, *Plant Simulation* cancels the **Loading Time** and considers it as completely elapsed. This means that the *Worker* carries at least one *part* to the *parts buffer*.

Note

The entrance of the *Workplace* is closed while its **Loading Time** is running as the *Worker* functions with reservations for the *Workplace*.

Note

Behavior during interruptions of the processing station

Failures of the processing station do not affect loading the *part* by the *Worker*. The *part* is already located at the *Worker* and loading takes place during the failure.

The same applies for processing stations which are **stopped**, **paused**, or **unplanned**.

Type

Attribute

Syntax

```
<Path>.LoadingTime:time
```

Assignment Value

You can assign a value of data type *time*.

Example

```
MyWorkplace.LoadingTime := 10:00
```

SimTalk

[UnloadingTime \[SimTalk\] - Workplace](#)

See also

[Loading Time \[Workplace\]](#)

[Maximum Dwell Time \[transport importer\]](#)

PickDropAtStore [SimTalk]

Sets what the *Worker* does when he walks to a *Workplace* attached to a *Store*, when picking up or dropping parts at this *Store*.

Type

Attribute

Syntax

```
<Path>.PickDropAtStore:string
```

Assignment Value

You can assign a value of data type *string*.

You can specify:

- "Walk to Workplace": Is the default setting that previous versions of *Plant Simulation* implicitly provided. The *Worker* walks to the *Workplace* to pick up parts or to drop parts.

For this setting, all settings of the *Workplace* are available.

Attributes	Times	Controls	User-defined
Station	MyStore ...		
	☒ At Entrance	☐	☒ At Exit
	Supported Services		☐
Capacity	1	☐	
	☒ Worker Stays Here After Completing the Job		☐
Distance in Queue	1	☐ m	
	☐ Walk to the End of the Animation Path of the Queue		☐
	☐ Walk Along the Animation Path of the Queue		☐
Pick/Drop at Store	Walk to Workplace		☐

Note

Plant Simulation only considers route computations with the methods *getRouteLength* and *getRouteCoordinates* for the setting **Pick/Drop at Store > Walk to Workplace**.

For the other settings of **Pick/Drop at Store** the method *getRouteLength* will return -1 and *getRouteCoordinates* will return an empty array.

- "Walk to Store Column": The *Worker* walks to the store column to pick up or to drop parts. The *Worker* can walk to the store column without any restrictions.

The screenshot shows a configuration window for 'PickDropAtStore'. It contains several fields and checkboxes:

- Capacity:** A text input field containing the value '-1'.
- Distance in Queue:** A text input field containing the value '1', followed by a unit selector set to 'm'.
- Worker Stays Here After Completing the Job:** A checked checkbox.
- Walk to the End of the Animation Path of the Queue:** An unchecked checkbox.
- Walk Along the Animation Path of the Queue:** An unchecked checkbox.
- Pick/Drop at Store:** A dropdown menu with 'Walk to Store Column' selected.

For this setting the following features are not available:

- You cannot enter a **Capacity**.
We assume the **Capacity** to be infinite.
- You cannot select **Worker Stays Here After Completing the Job**.
- You cannot enter the **Distance in Queue**.
- You cannot select **Walk to the End of the Animation Path of the Queue**.
- You cannot select **Walk Along the Animation Path of the Queue**.

Plant Simulation determines the locations to which the *Worker* walks by the dimensions of the side of the *Store* at which the *Workplace* is placed, by the number of columns at that side, and by the distance of the center point of the *Workplace* to the *Store* itself.

In addition, *Plant Simulation* uses the distance of the center point of the *Workplace* to the store front to place the pick or drop locations in front of the store column.

Walk to Store Column uses all pick or drop locations in front of the *Store* for computing the routes of the *Workers*. Be aware that large *Stores* with many columns will increase the complexity of the route computations and will impact performance.

- "Walk Along Store to Store Column": The *Worker* walks to the first or last column at the side of the *Store* where the *Workplace* is placed. Then the *Worker* extends his route and continues to walk along the *Store* to the store column to drop a part or pick up a part. Thus the routes to the store columns are restricted and each store column can only be reached from the previous or next store columns, except for the first and last store column. The *Worker* can walk to the first and last store columns without any restrictions.

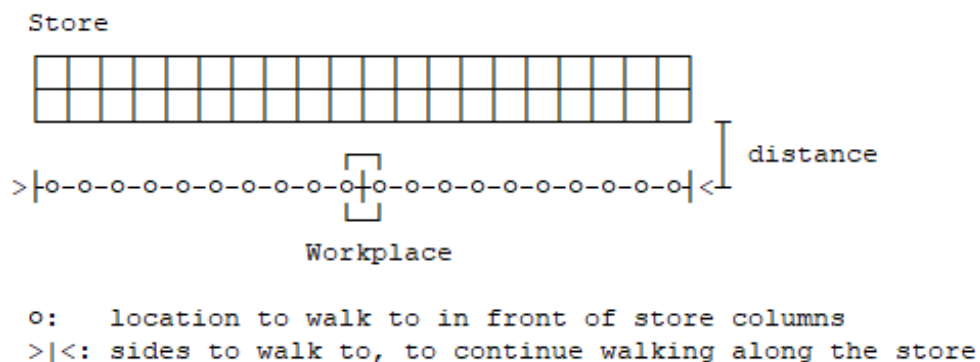
When the *Worker* extends his route from the first or last store column, he ignores all obstacles within his route.

Capacity	-1	
	☒ Worker Stays Here After Completing the Job	
Distance in Queue	1	m
	☐ Walk to the End of the Animation Path of the Queue	
	☐ Walk Along the Animation Path of the Queue	
Pick/Drop at Store	Walk Along Store to Store Column	

For this setting the following features are not available:

- You cannot enter a **Capacity**.
We assume the **capacity** to be infinite.
- You cannot select **Worker Stays Here After Completing the Job**.
- You cannot enter the **Distance in Queue**.
- You cannot select **Walk to the End of the Animation Path of the Queue**.
- You cannot select **Walk Along the Animation Path of the Queue**.
- *Plant Simulation* determines the locations to which the *Worker* walks by the dimensions of the side of the *Store* at which the *Workplace* is placed, by the number of columns at that side, and by the distance of the center point of the *Workplace* to the *Store* itself.

The diagram of the location to which the *Worker* walks looks like this:



Walk Along Store to Store Column does not increase the number of commonly reachable locations for the route calculation in the same way as the setting **Walk to Store Column** does. Thus, performance impact should be smaller.

Example

```
MyWorkplace.PickDropAtStore := "Walk to Store Column"
var behavior:string := MyWorkplace.PickDropAtStore // -> "Walk to Store Column"
```


SimTalk

[DistanceInQueue \[SimTalk\]](#)

[WalkAlongQueuePath \[SimTalk\]](#)

[WalkToQueuePathEnd \[SimTalk\]](#)

[WorkerStaysHere \[SimTalk\]](#)

See also

[Pick/Drop at Store](#)

[Model a Worker Picking Parts Up at/Placing Parts in a Store](#)

RecoveryTime [SimTalk]- Workplace

Sets the duration of the **Recovery Time** of the *Workplace* designated by <Path>.

Remarks

You can only define a **Recovery Time** for a *Workplace* that has a **Capacity** of **1**. The **Recovery Time** starts when a *Worker* leaves the *Workplace*.

The **Recovery Time** will not take effect, if you set the *Workplace* as the **Home Location** of a *Worker* and if the *Worker* wants to step onto this **Home Location/Workplace**.

Type

Attribute

Syntax

```
<Path>.RecoveryTime:time
```

Assignment Value

You can assign a value of data type *time*.

Example

```
MyWorkplace.RecoveryTime := 0.10
```

SimTalk

[HomeLocation \[SimTalk\] - Worker](#)

See also

[Home Location](#)

[Recovery Time \[Workplace\]](#)

[Capacity \[SimTalk\] - Workplace](#)

Station [SimTalk]

Sets the name of the **Station**, i.e., the material flow object, to which you want to assign the *Workplace* designated by <Path>.

Type

Attribute

Syntax

```
<Path>.Station:path
```

Assignment Value

You can assign a value of data type *path*.

Example

```
MyWorkplace.Station := MyStation
```

See also

[Station \[Workplace\]](#)

SupportedServices [SimTalk]

Sets or returns the list of the **Supported Services** of the *Workplace* designated by <Path>.

Remarks

The name is not case-sensitive, just like the names of *attributes* and *methods* of the objects are not case-sensitive.

To save memory and improve access speed, all places which are using such a case-insensitive string are pointing to the same string in main memory. The visible and unexpected result is that the first occurrence of the string defines how the string is written in terms of upper- and lower-casing.

In *SimTalk* you can compare strings in an case-insensitive manner with the `~=` operator, compare [Relational Operators](#).

Type

Attribute

Syntax

```
<Path>.SupportedServices -> string[]
```

Assignment Value

You can assign an *array* of data type *string* that contains the supported services.

Examples

```
var a : string[] := ["standard service", "drill"]
Workplace.SupportedServices := a
```

```
print MyWorkplace.SupportedServices
```

See also

[Supported Services \[button\]](#)

[Relational Operators](#)

UnloadCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

The **Unload Control** sets how the *Worker* places the part, which he carried to the *Workplace*, onto the attached *work station*.

Plant Simulation calls the **Unload Control** as soon as the *Worker* enters the *Workplace* attached to the target station.

The **standard unload control** as a *user-defined attribute* looks like this:

```
// workplace unload control ...
// ... to transfer all MUs from a worker on the workplace to the associated
// station
// @: worker
// ?: workplace
if @.failed = false and @.Pause = false
    // remember the station associated to the workplace
    var station:object := ?.station

    // get the first MU about to exit to repeat this until no MU wants to
    // exit anymore
    var mu:object := @.cont

    while mu /= void and station.full = false
        // wait until the entrance is free (for example for conveyors)
        waituntil station.EntranceFree

        // transfer the MU from the carrying worker to the station
        mu.move(station)

        // with configured unload time on workplace, wait for EndOfTime
        // event causing the transfer from worker to station
        waituntil mu.~ = station

        mu := @.cont
    end
end
```

Type

Attribute

Syntax

<Path>.UnloadCtrl:method

Assignment Value

You can assign a value of data type *method*.

Example

```
MyWorkplace.UnloadCtrl := &myUnloadControl
```

See also

Unload Control

UnloadingTime [SimTalk] - Workplace

Sets the **Unloading Time** at the *Workplace* designated by <Path>.

Remarks

The **Unloading Time** is the time it takes the *Worker* for placing the part, which he picked up at another station, onto target station.

If the *Worker* is to unload several *parts*, the **Unloading Time** accrues for each *part*.

You can use the anonymous identifier @ to access the *part* for which the unloading time applies.

Note

Behavior during interruptions of the Worker

Plant Simulation extends the **Unloading Time** by failure times of the *Worker*. The target processing station remains locked during the **failure**.

If the *Worker* **pauses** during the **Unloading Time**, *Plant Simulation* cancels unloading. The target processing station does not remain locked, and the *Worker* takes the *parts* to the *WorkerPool* with him. After the pause is over, unloading starts anew.

If the shift ends during the **Unloading Time**, *Plant Simulation* cancels the **Unloading Time** and the *Worker* carries the *part* to the *parts buffer*. The target processing station does not remain locked.

Note

Behavior during interruptions of the processing station through a failure, pause, etc.

The *Worker* only starts unloading when the target processing station can take on the *part* for processing. If this is not the case, *Plant Simulation* enters the *part* to be unloaded into the **Forward Blocking List**. The **Unloading Time** starts after the interruption is over.

The *Worker* continues unloading the *part* if the target processing station is being interrupted during the **Unloading Time**. *Plant Simulation* also enters the *part* to be unloaded into the **Forward Blocking List** if the target processing station cannot accept a *part* at the time of the actual unloading process (after the **Unloading Time** has elapsed). The *part* stays with the *Worker*. After the interruption has been removed, the *Worker* deposits the *part* at the target processing station without any time delay.

Type

Attribute

Syntax

```
<Path>.UnloadingTime:time
```

Assignment Value

You can assign a value of data type *time*.

Example

```
MyWorkplace.UnloadingTime := 10:00
```

SimTalk

[LoadingTime \[SimTalk\] - Workplace](#)

See also

[Unloading Time \[Workplace\]](#)

WalkAlongQueuePath [SimTalk]

Sets if the *Worker* **walks along the animation path of the queue** to the **end of the animation path of the queue** of the *Workplace* designated by <Path> (true) or not (false).

Remarks

When the *Worker* has reached the end of the **MU Animation Path**, *Plant Simulation* computes the *route* to the target, the **waiting position**, or to the *Workplace* along the **MU Animation Path**.

This setting is only relevant, if you also set *WalkToQueuePathEnd* to true.

Type

Attribute

Syntax

```
<Path>.WalkAlongQueuePath: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyWorkplace.WalkToQueuePathEnd := true  
MyWorkplace.WalkAlongQueuePath := true
```

SimTalk

[WalkToQueuePathEnd \[SimTalk\]](#)

See also

[Walk to the End of the Animation Path of the Queue](#)

WalkToQueuePathEnd [SimTalk]

Sets if the *Worker* **walks along the animation path of the queue** to the **end of the animation path of the queue** of the *Workplace* designated by <Path> (true) or not (false).

Remarks

WalkToQueuePathEnd is relevant, if you also set *WalkAlongQueuePath* to true.

Type

Attribute

Syntax

```
<Path>.WalkToQueuePathEnd: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyWorkplace.WalkToQueuePathEnd := true  
MyWorkplace.WalkAlongQueuePath := true
```

SimTalk

[WalkAlongQueuePath \[SimTalk\]](#)

See also

[Walk Along the Animation Path of the Queue](#)

WorkerStaysHere [SimTalk]

Sets if the *Worker* stays at the *Workplace* designated by <Path> after completing his job (**true**), or if he returns to the *WorkerPool* or to the **Home Location** if you configured one (**false**).

Remarks

You have to set the attribute to **false** if you activate the setting *GetJobOrdersAtHomeOnly* in the *WorkerPool*.

Note

A *Worker* who is not automatically brokered stays at the *Workplace* to which he was sent, no matter if you activated **Worker Stays Here After Completing the Job/WorkerStaysHere** or not in the *Workplace*.

Note

If a *Worker* stays on a *Workplace* while another *Worker* has to work on it, and if the **Capacity** of the *Workplace* is used up, *Plant Simulation* shoves the *Worker* aside who is not working. This *Worker* then exits the *Workplace* and returns to the *WorkerPool* or to the **Home Location** if you configured one.

Attribute

Type

Attribute

Syntax

```
<Path>.WorkerStaysHere: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyWorkplace.WorkerStaysHere := true
```

SimTalk

[GetJobOrdersAtHomeOnly \[SimTalk\]](#)

See also

[Worker Stays Here After Completing the Job](#)

[Get Job Orders At Home Only](#)

[Home Location](#)

FootPath

FootPath [object]

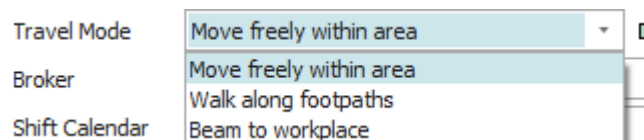
Use the object *FootPath* for modeling a path on which the *Worker* walks from the *WorkerPool* to the *Workplace*.

Image



Description

The *Worker-WorkerPool-Workplace-FootPath*-concept refines the *Broker-Importer-Exporter*-concept. In the **WorkerPool** you can select the **Travel Mode** with which the *Worker* gets to his work station:



- He can **walk freely within the area** of the simulation model, just like in the real world.
- He can be **teleported to the Workplace** you specify.
- He can **walk along the FootPaths** in the simulation model.

As a rule, the *Worker* functions as an **Exporter** with a capacity of 1. If you want to model and simulate walking distances between the *WorkerPool* and work stations or between work stations, use the *FootPath*. *Plant Simulation* animates the *Workers* on the *FootPath* if they move to the stations or in between the stations.

You can link several *FootPaths* with *Connectors* to create a network of *FootPaths*. The *Worker* walks within this network from his *WorkerPool* to the *Workplace*. Provided the *WorkerPool* and the *Workplace* are connected to the same network of *FootPaths*, the *Worker* walks from the *WorkerPool* to the *Workplace* and back on the shortest possible route.